HemoCare AI

AI-Assisted Anemia Screening, Blood Report Interpretation & Connected Healthcare Platform

Complete College Major Project Architecture & Code Guide

---

1. Project Overview & Clinical Objective

HemoCare AI is an end-to-end medical software platform engineered for preliminary automated screening of anemia and hematologic disorder subtype classification directly from Complete Blood Count (CBC) test parameters. It bridges the gap between lab test results and clinical action by delivering early AI risk alerts, personalized dietary guidance, doctor appointment scheduling, clinical consultation notes, and dynamic real-time medical notifications.

Key Innovations:

- **Dual-Inference Machine Learning Engine:** High-sensitivity binary screening combined with 9-class disorder subtype probability estimation.
- **Zero-False-Negative Tuning:** Task B decision threshold calibrated to 0.37 to guarantee 100% recall on anemic cases.
- **Connected Doctor-Patient Ecosystem:** Role-based workflows for Patients, Hematologists, and Healthcare Administrators.
- **Persistent Relational Storage:** Fully normalized MySQL database backing all screenings, clinical notes, availability schedules, notifications, and platform identity.

2. Full-Stack System Architecture

Layer	Technology	Key Responsibilities
Frontend Client	React 18, Vite, React Router 6, Recharts, Lucide Icons	Responsive SPA, clinical dashboards, assessment wizard, interactive notification bell, charts.
Backend Framework	Python 3.12+, Django 5.x, Django REST Framework (DRF)	REST API endpoints, PBKDF2 authentication, clinical data validation, ML model orchestrator.
Machine Learning	Scikit-Learn, Joblib, Random Forest, NumPy, Pandas	Task B binary screening model, Task A 9-class multiclass classifier, median imputation.
Relational Database	MySQL 8.0+ (InnoDB Engine, Database: anemia_system)	Patients, blood reports, ML outputs, consultation notes, doctor schedules, notifications.

3. Machine Learning Models & Clinical Metrics

The diagnostic core utilizes pre-trained Scikit-Learn Random Forest ensembles loaded from the backend/apps/ml_service/models/ directory:

A. Task B: Binary Anemia Screening (Healthy vs. Anemic)

- **Artifact:** task_b_scenario2_random_forest.joblib
  - **Decision Threshold: 0.37** (calibrated to prioritize patient safety over standard 0.50 cutoff)
  - **Clinical Recall / Sensitivity: 100.0%** (Zero false negatives in clinical validation)
  - **Specificity:** 100.0% | **ROC-AUC:** 1.0000 | **PR-AUC:** 1.0000

B. Task A: 9-Class Hematologic Disorder Diagnosis

- **Artifact:** task_a_scenario2_random_forest.joblib
  - **Predicted Classes:** (1) Healthy, (2) Iron deficiency anemia, (3) Normocytic normochromic anemia, (4) Normocytic hypochromic anemia, (5) Other microcytic anemia, (6) Macrocytic anemia, (7) Thrombocytopenia, (8) Leukemia, (9) Leukemia with thrombocytopenia.

C. 14 Complete Blood Count (CBC) Diagnostic Features:

1. **HGB**: Hemoglobin (g/dL) | 2. **RBC**: Red Blood Cell Count (x10■/µL) | 3. **HCT**: Hematocrit (%)
   4. **MCV**: Mean Corpuscular Volume (fL) | 5. **MCH**: Mean Corpuscular Hemoglobin (pg)
   6. **MCHC**: Mean Corpuscular Hemoglobin Conc. (g/dL) | 7. **RDW**: Red Cell Distribution Width (%)
   8. **WBC**: White Blood Cell Count (x10³/µL) | 9. **PLT**: Platelet Count (x10³/µL)
   10. **NEUT%** | 11. **LYMPH%** | 12. **MONO%** | 13. **EO%** | 14. **BASO%**

4. Backend Code & Key Files Breakdown

Backend directory: E:\mohini's program\.vscode\Anemia screening\

• screening/models.py — *Database Models (11 Tables)*

```
Defines all MySQL schemas using Django ORM:
• Patient: demographics (name, email, age, gender, weight, height)
• BloodReport: all 14 CBC test values + symptoms JSON
• ScreeningResult: binary predictions, probabilities, and raw JSON output
• DiagnosisReport: multiclass subtype classification & limitations
• Recommendation: evidence-based clinical actions (Diet, TIBC, Ferritin)
• Doctor: profile, specialization, experience, location, rating, status
• Appointment: patient consultation bookings, dates, times, reasons
• ConsultationNote: clinical diagnosis, prescription, dietary advice, follow-up
• Notification: dynamic reminders, alerts, deep links, read status
• PlatformSetting: system name, support email, contact phone, access policies
```

• screening/views.py — *API Controllers & Business Logic*

```
Implements DRF Generic API views and ViewSets:
• ScreenPatientAPIView (POST /api/screen-patient/): Unified screening pipeline
• RegisterAPIView (POST /api/auth/register/): Creates auth_user & Patient/Doctor
• LoginAPIView (POST /api/auth/login/): Validates credentials & returns user session
• NotificationViewSet (/api/notifications/): Role-based filtering & auto-seeding
• ConsultationNoteViewSet (/api/consultation-notes/): Doctor clinical notes CRUD
• PlatformSettingAPIView (/api/settings/): MySQL-backed platform settings
• PatientViewSet, DoctorViewSet, AppointmentViewSet: Standard CRUD operations
```

• screening/services.py — *Clinical Pipeline & Recommendations*

```
Core backend orchestrator:
• run_patient_screening_pipeline(): Validates physical ranges, stores Patient & BloodReport in MySQL,
executes ML Task B & Task A inference, stores DiagnosisReport, and triggers recommendations.
• generate_clinical_recommendations(): Generates medical follow-up plans (e.g. Iron + Vit C protocol
for microcytic anemia, B12 tests for macrocytic).
```

• screening/serializers.py — *REST Serialization & Form Mapping*

```
Maps JSON payloads to Django ORM models:
• ScreenPatientRequestSerializer: Accepts standard medical abbreviations or UI form field labels,
applying median imputation defaults for missing differential cell counts.
• ModelSerializers for Patient, BloodReport, Doctor, Appointment, ConsultationNote, Notification,
PlatformSetting.
```

• screening/urls.py — *REST API URL Routing*

```
Registers ViewSets on DefaultRouter and binds custom endpoints:
• /api/screen-patient/ -> ScreenPatientAPIView
• /api/auth/register/ & /api/auth/login/ -> Auth views
• /api/settings/ -> PlatformSettingAPIView
• /api/notifications/, /api/consultation-notes/, /api/doctors/, /api/appointments/
```

• config/settings.py — *Django Server Configuration*

```
Configures MySQL database connection (ENGINE: django.db.backends.mysql, NAME: anemia_system, USER:
root), CORS headers for React localhost:5173, installed apps, static files, and security middleware.
```

5. Frontend React Code & Architecture

Frontend directory: AI-Anemia-Screening\src\ (Vite + React Single Page Application)

• src/App.jsx — *Application Routes & Routing Table*

```
Defines all public and protected role-based routes with React Router 6:
• Public: / (Landing), /about, /how-it-works, /education, /login, /register
• Patient Portal: /patient/dashboard, assessment, result, reports, doctors, appointments, profile
• Doctor Portal: /doctor/dashboard, patients, appointments, consultation, availability, profile
• Admin Portal: /admin/dashboard, patients, doctors, verification, analytics, education, settings
```

• src/layouts/AppLayout.jsx — *Unified Role Layout & Dynamic Notification Bell*

```
Top navigation bar, role-based collapsible sidebar, user initials avatar, and dynamic notification
popover:
• Fetches live notifications from GET /api/notifications/?role={role}&email;={email}
• Numerical badge counter displaying unread count (e.g. '3 new')
• Interactive popover with category icons, 'Mark all read' button, dismiss button, and
click-to-navigate deep links.
```

• src/pages/patient/Patient.jsx — *Patient Portal Suite*

```
Comprehensive patient features:
• Assess(): Step-by-step CBC input wizard submitting to POST /api/screen-patient/
• Result(): Displays AI Screening risk badge, diagnostic interpretation, confidence score, dietary
guidance
• Doctors(): Searchable directory of verified hematologists with direct appointment booking
• Appts(): List of booked doctor consultations
```

• src/pages/doctor/Doctor.jsx — *Doctor Clinical Workspace*

```
Dedicated physician features:
• DDash(): Clinic dashboard with live patient statistics and quick clinical actions
• Notes(): Consultation notes manager saving clinical observations, prescriptions, and dietary advice
to MySQL
• DoctorAvailabilityPage(): Weekly consultation hours & time slot editor updating screening_doctor
• DoctorProfilePage(): Doctor credential and bio manager
```

• src/pages/admin/Admin.jsx — *Healthcare Administrator Portal*

```
System management and clinical governance:
• ADash(): Real-time analytics, screening trends, and appointment charts
• Verify(): Medical license review and approval workflow for new doctors
• AEdu(): Health education article editor & publishing system with category tags and MySQL/local
persistence
• ASettings(): Platform settings manager saving system name and policies to MySQL
```

6. MySQL Database: Tables & Schemas

Database Name: **anemia_system** (MySQL 8.0, Host: 127.0.0.1:3306)

Table Name	Key Columns	Description
`screening_patient`	`id, name, email, age, gender, weight, height`	Registered patient demographics.
`screening_bloodreport`	`id, patient_id, hgb, rbc, hct, mcv, mch, mchc, rdw, wbc, plt, neut, lymph, mono, eo, baso, symptoms`	14 CBC blood test values + symptom JSON.
`screening_screeningresult`	`id, report_id, prediction_label, confidence, raw_output`	Task B screening binary status & JSON.

screening_diagnosisreport	id, result_id, diagnosis_text, safety_limitations	Task A multiclass clinical subtype output.
screening_recommendation	id, report_id, category, recommendation_text, priority	Evidence-based clinical follow-up actions.
screening_doctor	id, name, email, spec, exp, loc, avail, rating, status	Physician profiles, weekly slots & status.
screening_appointment	id, patient_name, doctor_name, date, time, status, why	Booked consultations between patient & doctor.
screening_consultationnote	id, doctor_id, patient_id, notes, prescription, dietary_advice, follow_up	Clinical diagnosis notes & prescriptions.
screening_notification	id, role, recipient_email, title, message, type, is_read, link	Dynamic clinical reminders & system alerts.
screening_platformsetting	id, platform_name, support_email, contact_phone, institution, allow_patient_registration	Global platform identity & configuration.
auth_user	id, username, password (PBKDF2), email, first_name, is_staff, date_joined	Django security core for all user logins.

7. Clean Non-Overlapping SQL Queries Guide

Why do queries overlap in the terminal? Tables with 10+ columns or long text fields wrap across the terminal window, causing line breaks. Use the following proven techniques for crystal-clear results:

Trick 1: End queries with \G instead of semicolon ;

```
Using \G outputs records vertically in clean card format with zero text wrapping:
SELECT * FROM screening_bloodreport LIMIT 2\G
SELECT * FROM screening_screeningresult LIMIT 1\G
```

Trick 2: Select specific columns with string truncation:

```
-- 1. Registered Patients
SELECT id, name, email, age, gender, weight AS Weight_kg FROM screening_patient LIMIT 5;

-- 2. Screening Results & Predictions
SELECT id, prediction_label, ROUND(confidence, 3) AS confidence,
       raw_output->>'$.task_a.predicted_diagnosis' AS Specific_Diagnosis
FROM screening_screeningresult ORDER BY id DESC LIMIT 5;

-- 3. Booked Appointments
SELECT id, patient_name, doctor_name, date, time, status FROM screening_appointment;

-- 4. Doctor Consultation Notes (Truncating long text)
SELECT id, doctor_name, patient_name, date, LEFT(notes, 40) AS notes_summary,
       LEFT(prescription, 35) AS prescription FROM screening_consultationnote;

-- 5. Clinical Reminders & Notifications
SELECT id, role, title, type, is_read, link FROM screening_notification;

-- 6. Platform Settings
SELECT id, platform_name, support_email, contact_phone, updated_at FROM screening_platformsetting;
```

8. How to Run the Entire Project

Follow these simple steps in PowerShell / Command Prompt to launch the full system:

```
STEP 1: Prerequisites
• Python 3.12+ installed (with virtual environment in .\venv)
• Node.js 18+ and npm installed
• MySQL 8.0 server running on localhost:3306 with database 'anemia_system'

STEP 2: Start Django Backend Server (Terminal 1)
cd "E:\mohini's program\.vscode\Anemia screening"
.\venv\Scripts\activate
python manage.py migrate                # Applies all database migrations
python manage.py runserver 8000         # Starts backend at http://127.0.0.1:8000/

STEP 3: Start React Frontend Server (Terminal 2)
cd "E:\mohini's program\.vscode\Anemia screening\AI-Anemia-Screening"
npm install                             # Installs dependencies (if not installed)
npm run dev                             # Starts Vite dev server at http://localhost:5173/

STEP 4: Access the Application
• Public Portal:    http://localhost:5173/
• Patient Login:    Select 'Patient' role on /login (or register as Patient)
• Doctor Login:     Select 'Doctor' role on /login (or register as Doctor)
• Admin Portal:     Select 'Admin' role on /login
• Django Admin:     http://127.0.0.1:8000/admin/ (Superuser dashboard)
```